

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 20%

Read the given scenario carefully and analyze the problem using appropriate computer vision techniques. Justify your answers with relevant reasoning and comparisons wherever necessary.

1. Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent.

Analyze the issue and explain how preprocessing improves feature detection performance.

Scenario:

A security camera system is used to identify and recognize vehicles entering a parking area. Images are captured at different times of the day, such as morning, afternoon, and night. Due to changes in sunlight, shadows, and artificial lighting, the appearance of the vehicles changes significantly.

Problem:

When feature detection is performed directly on the raw images, the system detects different features for the same vehicle. In bright images, some regions become overexposed, while in dark images, important edges and corners may disappear. Noise and shadows can also produce false features.

Analysis:

For example, a car captured in the afternoon may have clearly visible edges and corners. The same car captured at night may have low contrast, making these features difficult to detect. Therefore, feature detectors such as **SIFT, ORB, or Harris Corner Detector** may produce inconsistent results.

Preprocessing Solution:

Before feature detection, the images are preprocessed using:

1. **Grayscale conversion** – reduces unnecessary color information.
2. **Noise removal** – Gaussian or median filtering removes unwanted noise.

3. **Contrast enhancement** – histogram equalization or CLAHE improves visibility.
4. **Brightness normalization** – reduces the effect of different lighting conditions.
5. **Sharpening/edge enhancement** – makes important boundaries more distinct.

Result:

After preprocessing, the vehicle images have more uniform brightness and improved contrast. Important corners, edges, and keypoints can be detected more consistently. This improves feature matching and increases the accuracy of vehicle recognition.

Conclusion:

This case study shows that preprocessing is essential when images are captured under varying lighting conditions. By reducing noise, normalizing illumination, and enhancing contrast, preprocessing produces stable and reliable features, resulting in better performance of the overall computer vision system.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching.

Analyze why edge detection is insufficient and justify the use of corner detection in this case.

Scenario:

A computer vision system is designed to identify and match objects in images. The image contains **straight edges, corners, and textured regions**. The system uses an edge detector such as the **Canny Edge Detector**, but it fails to find enough reliable key points for image matching.

Problem:

Edge detection identifies locations where the image intensity changes significantly. However, an edge mainly provides information in **one direction**. A long straight edge may contain many pixels, but it does not provide a unique location for matching.

For example, the side of a rectangular object may produce a strong edge, but every point along that edge looks similar. Therefore, the system cannot easily determine the exact corresponding point in another image.

Analysis:

- **Edge detection** finds boundaries and contours.
- It is good for detecting the **shape or outline** of objects.
- However, straight edges contain little unique information for point-to-point matching.
- Textured regions may also produce many small edges, making matching unreliable.
- Edge detectors do not specifically identify distinctive points such as intersections or sharp changes in direction.

Why Corner Detection is Better:

Corner detection identifies points where intensity changes significantly in **multiple directions**. These points are more distinctive and can be used as reliable key points.

For example, consider a **book**:

- The four sides of the book create straight edges.
- The four corners are distinctive points.
- A corner remains easier to identify when the image is moved, rotated, or matched with another image.

Algorithms such as **Harris Corner Detector** and **Shi-Tomasi Corner Detector** can identify these distinctive points.

Comparison:

Edge Detection	Corner Detection
Detects boundaries	Detects distinctive points
Works mainly along one direction	Detects changes in multiple directions

Edge Detection

Produces continuous lines

Less suitable for point matching

Useful for object boundaries

Corner Detection

Produces specific key points

Better for feature matching

Useful for recognition and tracking

Result:

By replacing or combining edge detection with corner detection, the system obtains more distinctive key points from the image. These points can then be matched between images more accurately.

Conclusion:

Edge detection alone is insufficient for reliable feature matching because straight edges do not provide unique points. **Corner detection is preferred because corners contain strong intensity changes in multiple directions and provide distinctive, repeatable key points,** making them more suitable for image matching, object recognition, and tracking.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results.

Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Scenario:

A road-monitoring system uses the **Hough Transform** to detect lane lines from camera images. However, the captured images contain noise caused by poor lighting, rain, camera vibration, and low image quality. As a result, the system detects incorrect or extra lines.

Problem:

The Hough Transform detects lines by identifying points that vote for possible lines in **Hough space**. Noise introduces unwanted edge points. These noisy points also cast votes, creating false peaks in the accumulator.

Analysis of the Effect of Noise:

- Noise creates **false edges** in the image.
- False edges generate unnecessary votes in Hough space.
- Weak real lines may not receive enough votes to be detected.
- Strong noise can produce **multiple incorrect lines**.
- The detected line position and angle may become inaccurate.
- More noise increases computational complexity because many unnecessary points are processed.

Example:

Suppose a road image contains two actual lane lines. Due to road-surface noise and rain, many small edges are detected. The Hough Transform may interpret these small edges as additional lines, producing several incorrect lane boundaries.

Improvements

1. Preprocessing

Apply preprocessing before the Hough Transform:

- **Gaussian/Median filtering:** Reduces image noise.
- **Grayscale conversion:** Simplifies the image.
- **Contrast enhancement:** Makes important lines more visible.
- **Canny edge detection:** Extracts meaningful edges before Hough processing.
- **Morphological operations:** Remove small unwanted regions and connect broken edges.

2. Parameter Selection

The accuracy can also be improved by selecting suitable Hough parameters:

- **Higher threshold:** Removes weak false lines caused by noise.

- **Lower threshold:** Useful when real lines are weak, but may increase false detections.
- **Appropriate θ resolution:** Controls the precision of line angles.
- **Appropriate ρ resolution:** Controls the precision of line positions.
- **Minimum line length:** Rejects very short lines caused by noise.
- **Maximum line gap:** Helps connect broken parts of the same actual line.

Result

After applying noise reduction and suitable parameter tuning, the number of false lines decreases. The actual lane lines receive stronger and more consistent votes, allowing the system to detect them more accurately.

Conclusion:

Noise significantly affects the Hough Transform by creating false edge points and incorrect accumulator peaks. **Preprocessing combined with proper threshold, line length, angle, and gap parameters** improves line detection accuracy and makes the system more robust in noisy environments.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively.

Analyze why scale invariant methods perform better in this scenario.

Scenario:

A computer vision system is used to identify the same object from two images. In the first image, the object is captured from a normal distance and orientation. In the second image, the object is **smaller/larger and rotated**. Traditional feature matching fails to correctly match the images, while a **scale-invariant method such as SIFT** performs successfully.

Problem:

Traditional feature detection methods often depend on the original **size, orientation, and appearance** of the object. When the object is resized or rotated, the detected features may change significantly. Therefore, corresponding points between the two images cannot be matched reliably.

Analysis

Consider a **mobile phone** photographed from two different positions:

- Image 1: Phone appears large and is horizontally oriented.
- Image 2: Phone appears smaller and is rotated by 45°.

A traditional feature detector may identify different key points because the size and orientation have changed.

Scale-invariant methods solve this problem by detecting features that remain distinctive even when the image undergoes transformations.

Why Scale-Invariant Methods Perform Better

1. Scale Invariance

Methods such as **SIFT** detect features at different scales using a scale-space representation. Therefore, the same feature can be recognized even if the object becomes larger or smaller.

2. Rotation Invariance

The feature descriptor is assigned an orientation based on the local image structure. Thus, a rotated object can still produce a similar descriptor.

3. Robust Feature Descriptors

SIFT creates descriptors based on local gradients around key points. These descriptors are more distinctive than simply comparing pixel positions.

4. Better Correspondence

The same physical feature produces similar descriptors in both images, allowing reliable matching despite changes in size and orientation.

Example

A corner of a mobile phone may appear at:

- **Large scale:** 50×50 pixels
- **Small scale:** 25×25 pixels
- **Different orientation:** Rotated by 45°

Traditional matching may consider these different features. A scale- and rotation-invariant method recognizes them as the **same underlying feature**.

Result

Traditional Matching	Scale-Invariant Matching
Sensitive to scale changes	Handles scale changes
Sensitive to rotation	Handles rotation
Fewer reliable matches	More reliable matches
May produce incorrect matches	Produces better correspondence
Suitable for limited transformations	Suitable for varying transformations

Conclusion:

Scale-invariant feature matching performs better because it is designed to detect and describe features **independently of image scale and orientation**. This makes methods such as **SIFT** highly effective for object recognition, image stitching, tracking, and matching images captured from different viewpoints.

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy.

Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

Scenario:

A computer vision system is used for **face recognition**. The system extracts hundreds or thousands of features from each face image, such as edges, textures, shapes, and pixel information. Processing all these features requires high memory and computational time.

Problem:

Many extracted features may contain **similar or redundant information**. For example, several features may describe the same facial region. Using all features increases computational cost and can reduce the efficiency of the recognition system.

Analysis

Principal Component Analysis (PCA) is a dimensionality-reduction technique that transforms a large set of correlated features into a smaller set of **uncorrelated principal components**.

PCA works by:

1. **Collecting the original features**
 1. The system may initially have 1,000 features.
2. **Finding correlations**
 1. PCA identifies features that contain similar information.
3. **Calculating principal components**
 1. New components are created that capture the most important variations in the data.
4. **Ranking components**
 1. Components are arranged according to the amount of variance they preserve.
5. **Selecting important components**
 1. The system may reduce 1,000 features to only 100 important components.

Example

Suppose a face recognition system extracts **1,000 features**.

Without PCA:

1,000 features → Classification → High computation

With PCA:

1,000 features → PCA → 100 important components → Classification → Lower computation

The 100 components retain most of the useful information while removing redundant features.

Benefits of PCA

- **Reduces dimensionality:** Fewer features need to be processed.
- **Removes redundancy:** Correlated information is combined.
- **Reduces computational cost:** Algorithms work faster.
- **Reduces memory usage:** Smaller feature representations require less storage.
- **Improves efficiency:** Classification and matching become faster.
- **Can reduce noise:** Less important variations can be discarded.

Result

After applying PCA, the system processes a much smaller feature set while retaining most of the important information. This results in **faster processing, lower memory requirements, and more efficient feature matching or classification.**

Conclusion:

PCA improves feature optimization by converting a large number of correlated features into a smaller number of meaningful principal components. Therefore, it is useful for reducing **dimensionality, redundancy, computational cost, and processing time** while preserving the most important information in the dataset.

Code of Conduct:

I DHAANUSH RAJAN(192421162) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts

Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand